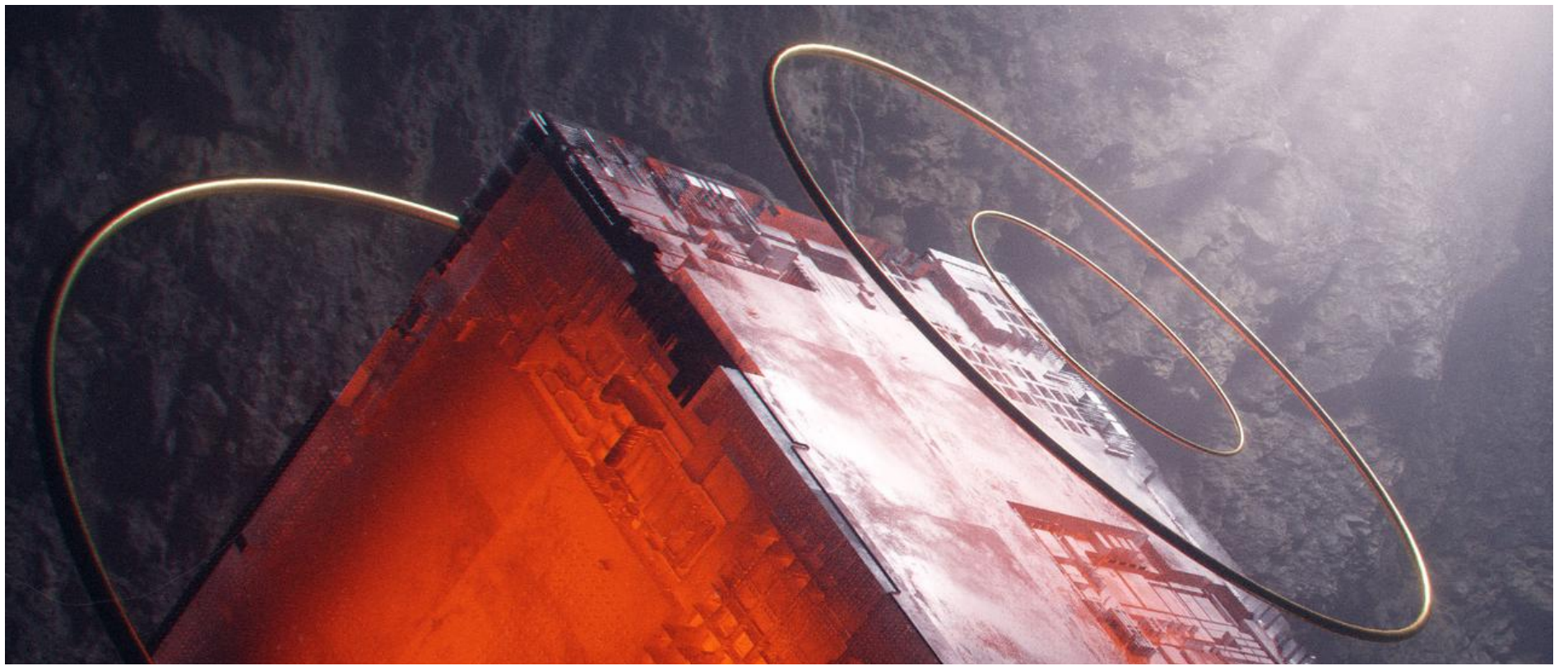




25年下半年面试真题预测及解析

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)
- [📖 618专题--大促搭投实践&高可用性保障](#)
- [📖 年中复盘会（1/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 年中复盘会（2/2） —— 25年上半年中大厂高频面试总结](#)

2. 真题预测解析

2.1 攻击

2.1.1 SQL注入

攻击者通过在输入字段中注入恶意 SQL 代码，从而篡改数据库查询。

防御措施：

- **使用参数化查询**：通过预编译的 SQL 语句和参数化查询，避免直接拼接 SQL。
- **ORM 框架**：使用 ORM 框架（如 Hibernate、Entity Framework）来处理数据库操作。
- **输入验证**：对用户输入进行严格的验证，确保输入符合预期格式。
- **Web 应用防火墙（WAF）**：部署 WAF 来自动检测和过滤恶意 SQL 注入请求。

2.1.2 XSS（Cross-site scripting）

攻击通过在网页中注入恶意脚本，当这些脚本在用户的浏览器中执行时，可能会窃取用户数据或执行恶意操作。

- 存储型 XSS
 - 注入型脚本永久存储在目标服务器上。当浏览器请求数据时，脚本从服务器上传并执行。
- 反射型 XSS
 - 当用户点击一个恶意链接，或者提交一个表单，或者进入一个恶意网站时，注入脚本进入被攻击者的网站。Web 服务器将注入脚本，比如一个错误信息，搜索结果等返回到用户的浏览器上。由于浏览器认为这个响应来自"可信任"的服务器，所以会执行这段脚本。
- 基于 DOM 的 XSS
 - 通过修改原始的客户端代码，受害者浏览器的 DOM 环境改变，导致有效载荷的执行。也就是说，页面本身并没有变化，但由于 DOM 环境被恶意修改，有客户端代码被包含进了页面，并且意外执行

防御措施：

- **输入验证与净化**：对用户输入进行严格的验证和净化，移除潜在的危险字符。
- **输出编码**：在将数据插入 HTML、JavaScript、CSS 或 URL 中时，进行适当的编码。
- **设置 HttpOnly Cookie**：将重要 Cookie 设置为 HttpOnly，防止 JavaScript 访问。

2.1.3 CSRF（Cross-site request forgery）跨站请求伪造

攻击者诱导受害者进入第三方网站，在第三方网站中，向被攻击网站发送跨站请求。利用受害者在被攻击网站已经获取的注册凭证，绕过后台的用户验证，达到冒充用户对被攻击的网站执行某项操作的目的。

- 一个典型的CSRF攻击有着如下的流程：
 - 受害者登录a.com，并保留了登录凭证（Cookie）。
 - 攻击者引诱受害者访问了b.com。
 - b.com 向 a.com 发送了一个请求：a.com/act=xx。浏览器会默认携带a.com的Cookie。
 - a.com接收到请求后，对请求进行验证，并确认是受害者的凭证，误以为是受害者自己发送的请求。
 - a.com以受害者的名义执行了act=xx。
 - 攻击完成，攻击者在受害者不知情的情况下，冒充受害者，让a.com执行了自己定义的操作。
- 处理：cookie samesite

2.2 Cookie

2.2.1 Secure 和 HttpOnly 属性

Secure 属性和HttpOnly 属性

- Secure：表示只应通过被 HTTPS 协议加密过的请求发送给服务端；
- HttpOnly：JavaScript Document.cookie API 无法访问带有 HttpOnly 属性的 cookie；此类 Cookie 仅作用于服务器。例如，持久化服务器端会话的 Cookie 不需要对 JavaScript 可用，而应具有 HttpOnly 属性。

2.2.2 SameSite

SameSite 可以有下面三种值：

- 1. None：浏览器会在同站请求、跨站请求下继续发送 cookies，不区分大小写；
- 2. Strict：浏览器将只在访问相同站点时发送 cookie；
- 3. Lax：与 Strict 类似，但用户从外部站点导航至 URL 时（例如通过链接）除外。在新版本浏览器中，为默认选项，Same-site cookies 将会为一些跨站子请求保留，如图片加载或者 frames 的调用，但只有当用户从外部站点导航到 URL 时才会发送。如 link 链接；

大多数主流浏览器基本上已经将 SameSite 的默认值迁移至 Lax。如果想要指定 Cookies 在同站、跨站请求都被发送，现在需要明确指定 SameSite 为 None；

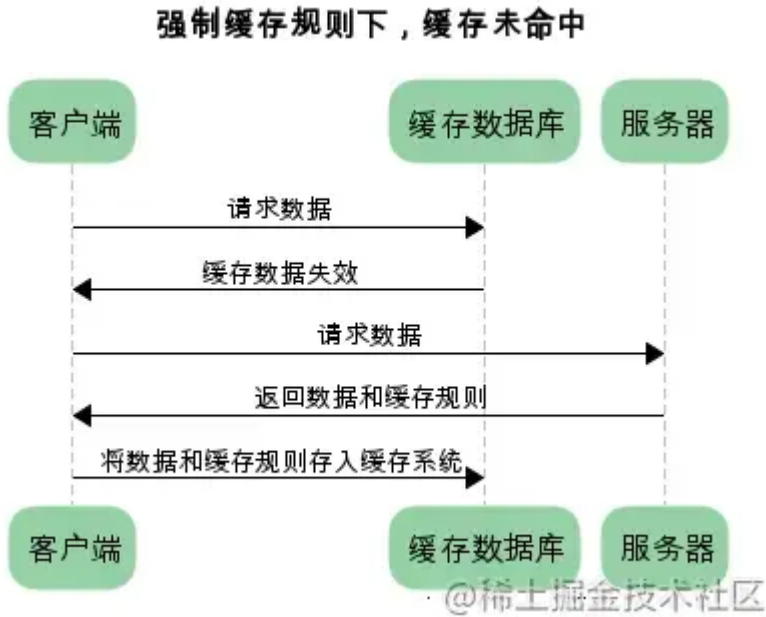
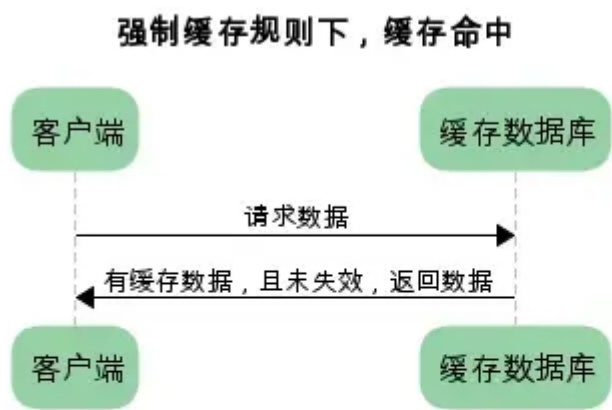
2.2.3 安全性

- 1. 使用 HttpOnly 属性可防止通过 JavaScript 访问 cookie 值；
- 2. 用于敏感信息（例如指示身份验证）的 Cookie 的生存期应较短，并且 SameSite 属性设置为 Strict 或 Lax

2.3 缓存

2.3.1 强制缓存

强制缓存不需要与服务器发生交互。



Expires

HTTP 1.0 的字段，表示缓存到期时间，是一个绝对的时间 (当前时间+缓存时间)。在响应消息头中，设置这个字段之后，就可以告诉浏览器，在未过期之前不需要再次请求。

缺点：若修改电脑的本地时间，会导致浏览器判断缓存失效 这里修重新修改缓存

Cache-control

HTTP/1.1

- 1. no-cache：告诉浏览器忽略资源的缓存副本，强制每次请求直接发送给服务器，拉取资源，但不是“不缓存”，相当于需要使用协商缓存，禁止使用强制缓存；
- 2. no-store：强制缓存在任何情况下都不要保留任何副本，相当于不使用强制缓存和协商缓存；
- 3. public 任何路径的缓存者（本地缓存、代理服务器），可以无条件的缓存改资源，不设置默认为public；
- 4. private 只针对单个用户或者实体（不同用户、窗口）缓存资源；

2.3.2 对比缓存

last-modified

- 1. 服务器在响应头中设置last-modified字段返回给客户端，告诉客户端资源最后一次修改的时间；

```
a. Last-Modified: Sat, 30 Mar 2029
05:46:11 GMT
```

Etag

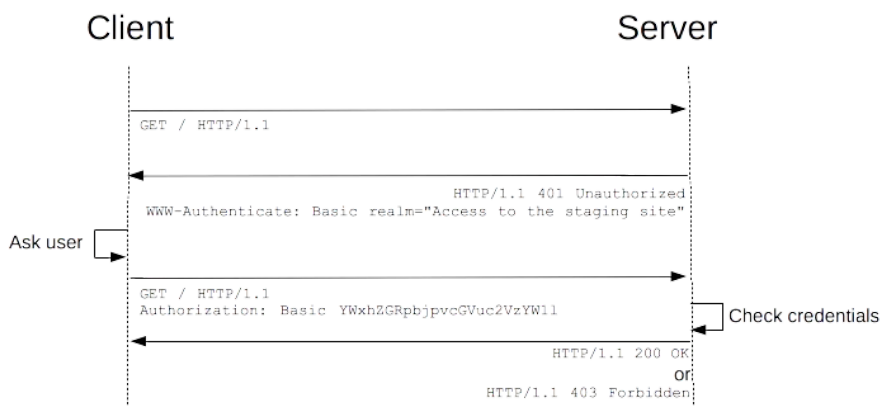
tag是根据文件内容，算出一个唯一的值。服务器存储着文件的 Etag 字段。

If-None-Match。

- 2. 浏览器在这个值和内容记录在浏览器的缓存数据库中；
- 3. 下次请求相同资源，浏览器将在请求头中设置if-modified-since的值（这个值就是第一步响应头中的Last-Modified的值）传给服务器；
- 4. 服务器收到请求头的if-modified-since的值与last-modified的值比较，如果相等，表示未进行修改，则返回状态码为304；如果不相等，则修改了，返回状态码为200，并返回数据；

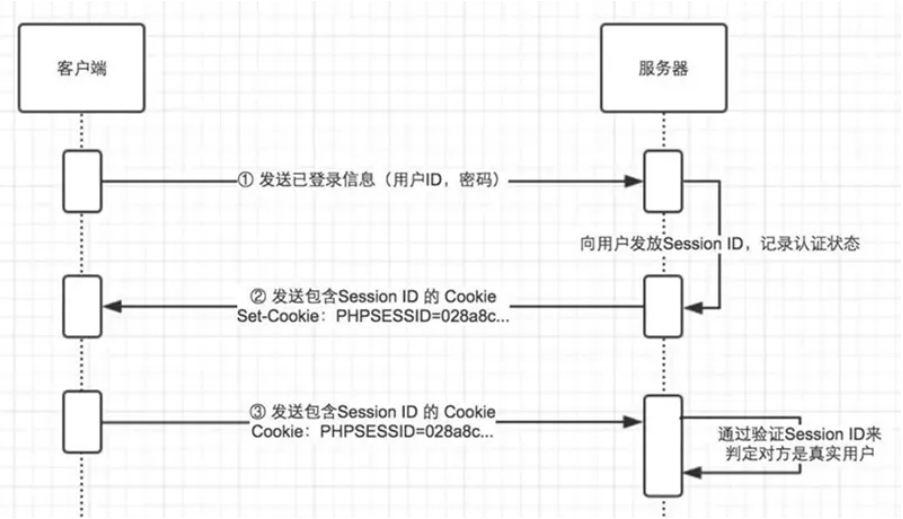
2.4 鉴权

HTTP Basic Authentication



服务器同样进行比较，命中返回 304, 不命中返回新资源和 200。Etag 的优先级高于 Last-Modified

session-cookie

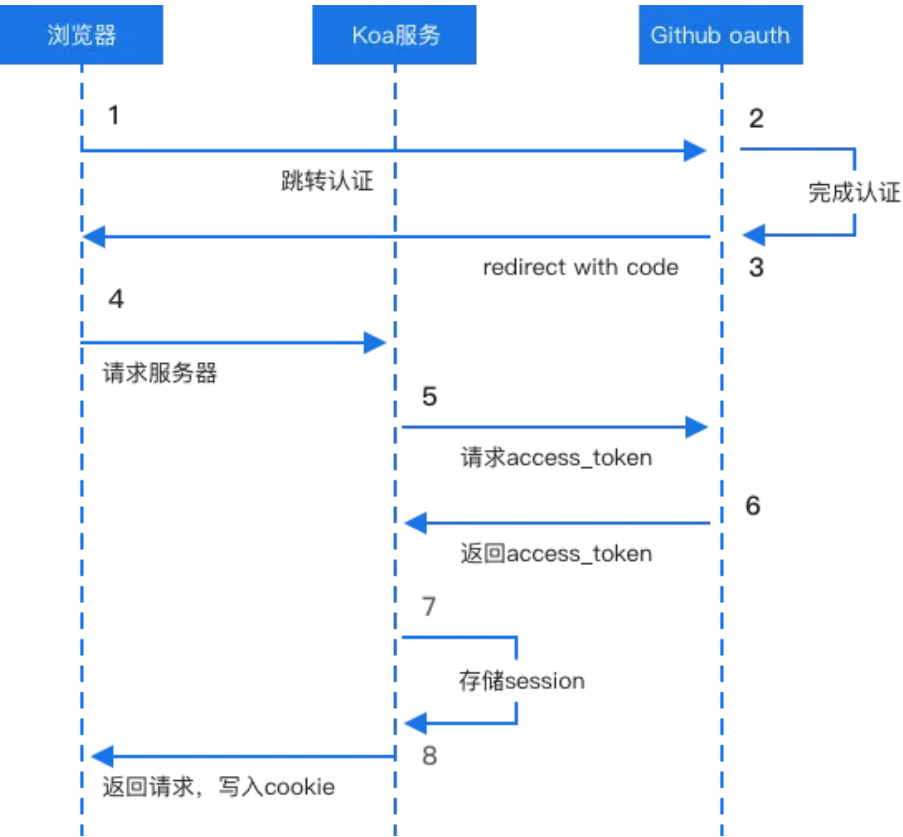


Token JWT (JSON Web Token)

AI智能助手 使用 Authorization

- 1. 客户端使用用户名跟密码请求登录；
- 2. 服务端收到请求，去验证用户名与密码；
- 3. 验证成功后，服务端会签发一个 Token，再把这个 Token 发送给客户端；
- 4. 客户端收到 Token 以后可以把它存储起来，比如放在 Cookie 里或者Local Storage 里；
- 5. 客户端每次向服务端请求资源的时候需要带着服务端签发的 Token；
- 6. 服务端收到请求，然后去验证客户端请求里面带着的 Token（request头部添加Authorization），如果验证成功，就向客户端返回请求的数据，如果不成功返回401错误码，鉴权失败

OAuth



跟cookie-session 区别

cookie-session

- 1. 认证方式局限于在浏览器中使用，cookie 是浏览器端的机制，如果在app端就无法使用 cookie；
- 2. 为了满足全局一致性，我们最好把 session 存储在 redis 中做持久化，而在分布式环境下，我们可能需要在每个服务器上都备份，占用了大量的存储空间；

3. 在不是 Https 协议下使用 cookie ，容易受到 CSRF 跨站点请求伪造攻击。

token的缺点：

- 1. 加密解密消耗使得 token 认证比 session-cookie 更消耗性能；
- 2. token 比 sid 大，更占带宽；

Encoded

PASTE A TOKEN HERE

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4uRG91IiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

{

"alg": "HS256",

"typ": "JWT"

}

PAYLOAD: DATA

{

"sub": "1234567890",

"name": "John Doe",

"iat": 1516239822

}

VERIFY SIGNATURE

HMACSHA256(

base64UrlEncode(header) + "." +

base64UrlEncode(payload),

your-256-bit-secret

)

☐ secret base64 encoded

2.5 DNS

1. DNS：将主机名转为具体IP的服务

根域：就是一个“.”号，由Internet名字注册授权机构管理。全世界共有13台根域服务器。

顶级域：由Internet名字授权机构管理。共有3种类型的顶级域：

- 组织域：采用3个字符的代号，如edu、com、gov等。
- 地理域：采用2个字符的国家/地区代号，如cn、jp、hk等。
- 反向域：这是一个特殊域，名称为in-addr.arpa，用于将IP映射到域名。

二级域：注册到个人、组织或公司的名称。二级域下还可以创建子域。

主机名：就是FQDN最左边的部分。使用“hostname”命令可以显示当前主机名。

2. 操作过程：递归查询，优先查找是否有DNS缓存

```
graph LR
    Client[客户端]
    Local[本地DNS服务器]
    Root[根域名服务器]
    TLD[顶级域名服务器]
    Auth[权威域名服务器]

    Client -- 1 --> Local
    Local -- 2 --> Root
    Root -- 3 --> Local
    Local -- 4 --> TLD
    TLD -- 5 --> Local
    Local -- 6 --> Auth
    Auth -- 7 --> Local
    Local -- 8 --> Client
```

2.6 CDN

<https://help.aliyun.com/zh/cdn/what-is-a-dns-cname-record?spm=a2c4g.11186623.0.0.273c2d49OzLtqd>

不使用CDN

- 1. 用户在自己的浏览器中输入要访问的网站域名。
- 2. 浏览器向 本地DNS服务器 请求对该域名的解析。

使用CDN

- 1. DNS 服务器根据用户 IP 地址，将域名解析成相应节点的缓存服务器IP地址，实现用户就近访问

3. 本地DNS服务器中如果缓存有这个域名的解析结果，则直接响应用户的解析请求。
4. 本地DNS服务器中如果没有关于这个域名的解析结果的缓存，则以递归方式向整个DNS系统请求解析，获得应答后将结果反馈给浏览器。
5. 浏览器得到域名解析结果，就是该域名相应的服务设备的 IP 地址。
6. 浏览器向服务器请求内容。
7. 服务器将用户请求内容传送给浏览器。

具体访问CDN过程

1. 用户向浏览器输入www.web.com这个域名，浏览器第一次发现本地没有DNS缓存，则向网站的DNS服务器请求；
2. 网站的DNS域名解析器设置了CNAME，指向了www.web.51cdn.com,请求指向了CDN网络中的智能DNS负载均衡系统；
3. 智能DNS负载均衡系统解析域名，把对用户响应速度最快的IP节点（CDN服务器）返回给用户；
4. 用户向该IP节点（CDN服务器）发出请求；
5. 由于是第一次访问，CDN服务器会向原web站点请求，并缓存内容；

2.7 WebSocket

Code block

```
1  class AIChatSocket {
2      private socket: WebSocket;
3      private messageQueue: string[] = [];
4
5      constructor(token: string) {
6          // 建立安全连接
7          this.socket = new WebSocket(`wss://ai-service.com/chat?token=${token}`);
8
9          this.socket.onmessage = (event) => {
10             const { content, is_end } = JSON.parse(event.data);
11
12             // 实时处理流式响应
13             if (content) {
14                 this.onStreamResponse(content);
15             }
16
17             if (is_end) {
18                 this.onComplete();
19             }
20         };
21
22         this.socket.onclose = () => {
23             // 断线自动重连
24             setTimeout(() => this.reconnect(), 3000);
25         };
26     }
27
28     sendMessage(text: string) {
29         if (this.socket.readyState === WebSocket.OPEN) {
30             this.socket.send(JSON.stringify({
31                 message: text,
32                 temperature: 0.7
33             }));
```

```
34     } else {
35         this.messageQueue.push(text);
36     }
37 }
38
39 private reconnect() {
40     // 重连逻辑...
41 }
42 }
```

- 1. 增量更新渲染
- 2. 心跳检测
- 3. 断线重连

Code block

```
1  function useAIChat() {
2      const [answer, setAnswer] = useState("");
3
4      useEffect(() => {
5          const socket = new WebSocket(API_URL);
6
7          // 增量更新回答
8          socket.onmessage = (event) => {
9              setAnswer(prev => prev + event.data);
10         };
11
12         // 心跳检测
13         const heartbeat = setInterval(() => {
14             socket.send(JSON.stringify({ type: "ping" }));
15         }, 30000);
16
17         return () => {
18             clearInterval(heartbeat);
19             socket.close();
20         };
21     }, []);
22 }
```

2.8 Serverless

1. Serverless 架构

Serverless 是一种云计算模式，允许开发者无需管理服务器即可构建和部署应用程序。它主要由两部分组成：

- **FaaS (Function as a Service)**：即函数即服务，允许开发者编写事件驱动的函数，并由云平台负责运行和管理。
- **BaaS (Backend as a Service)**：即后端即服务，提供数据库、对象存储、消息队列等后端服务。

2. BFF (Backend For Frontend)

BFF 是一种架构模式，全称为 **Backend For Frontend**，即“服务于前端的后端”。它的主要作用是：

- **API 聚合**：将多个后端微服务的数据进行聚合、裁剪和格式化，为前端提供统一的 API。
- **用户体验适配**：根据不同前端（如 Web、移动端）的需求，提供定制化的数据接口。
- **减少沟通成本**：前端和后端团队可以更独立地开发，减少因接口设计而产生的沟通成本

2.9 Restful API

REST (Representational State Transfer) 即“表现层状态转换”，是一种软件架构风格，用于设计网络应用程序的接口。

RESTful API 遵循以下原则：

- **无状态性**：每个请求都包含处理该请求所需的所有信息，服务器不存储客户端状态。
- **统一接口**：有一套统一的接口，使系统间交互更简洁。
- **资源驱动**：数据和功能被视为资源，通过 URI 唯一标识。
- **支持多种表示形式**：资源可用多种格式表示，如 JSON、XML，其中 JSON 最流行。
- **分层系统**：允许通过不同层次组合构建系统

Code block

```
1  GET  /users      →  获取用户列表
2  POST /users      →  创建新用户
3  PUT  /users/001  →  全量更新用户
```

与传统的 RPC 风格 API 相比，RESTful 更强调通过标准协议实现资源操作而非功能调用。

2.10 GraphQL

1. 查询语言

客户端精确指定需要的数据结构：

Code block

```
1  query {
2    user(id: "123") {
3      name
4      orders {
5        date
6        total
7      }
8    }
9  }
```

2. 类型系统

强类型 Schema 定义（示例）：

Code block

```
1  type User {
2    id: ID!
3    name: String!
4    email: String
5    orders: [Order]
6  }
7
8  type Order {
9    id: ID!
10   date: String!
11   total: Float
12 }
```

3. 执行引擎

通过 resolver 函数实现数据聚合：

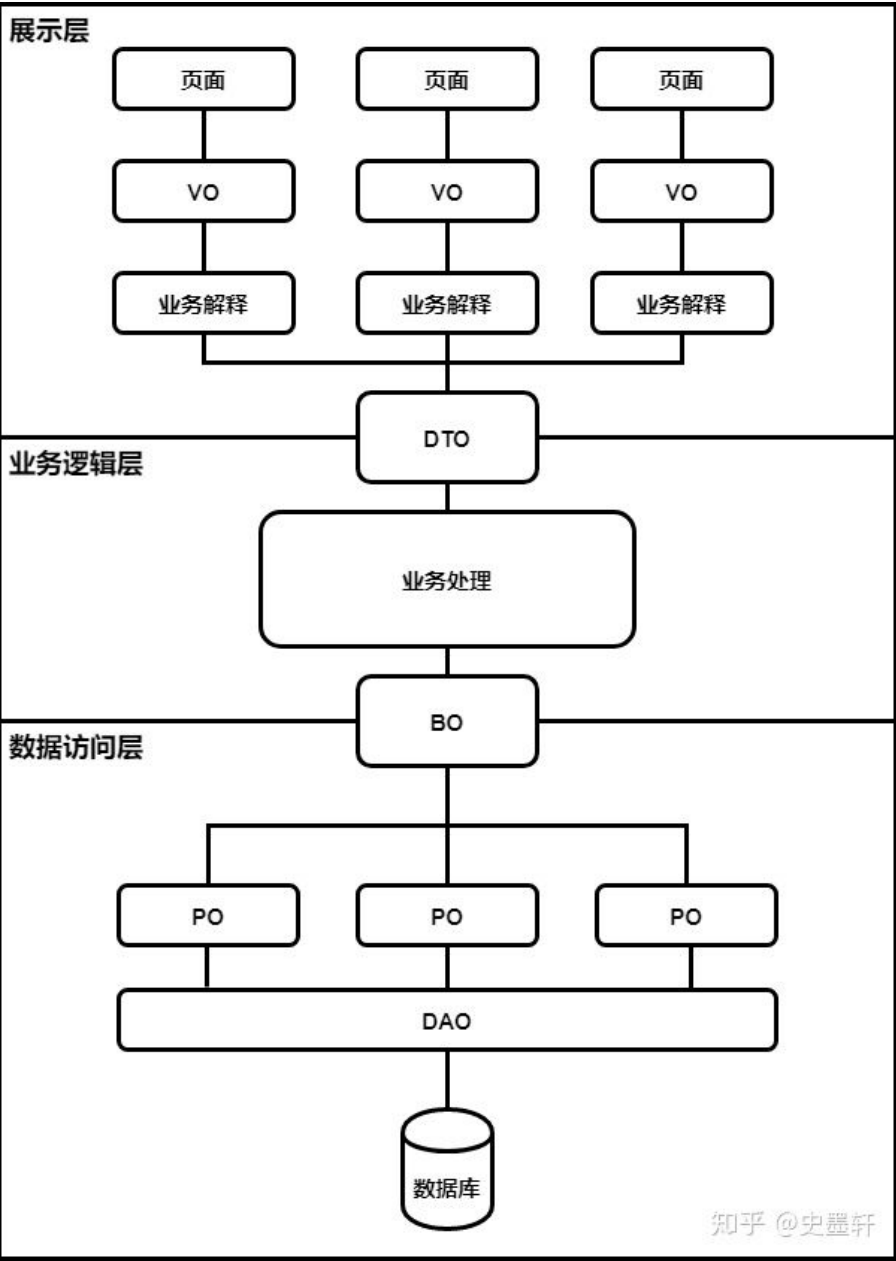
Code block

```
1  const resolvers = {
2    Query: {
```



```
3     user: (parent, args) => getUser(args.id)
4   },
5   User: {
6     orders: (user) => getOrdersByUser(user.id)
7   }
8 };
```

2.11 前后端联调规范



BO：Business Object 业务对象

PO：Persistent Object，PO 的字段与数据库表的字段严格对应

DAO：Data Access Object 数据访问对象，DAO 主要负责与数据库进行交互，封装了对数据的增、删、改、查（CRUD）操作。它通常与 PO（持久化对象）一起使用，通过操作 PO 来实现对数据库的访问

2.12 AI流式传输

1. HTTP 流式传输
2. Server-Sent Events (SSE)
3. Websocket
- 设置响应头 `Transfer-Encoding: chunked`
- 数据以 `text/event-stream` 格式传输
- 服务器可以将数据分块发送给客户端。客户端逐块接收并处理数据。
- 前端通过 `eventSource` 解析

2.13 打开url访问完整流程

1. DNS 解析

- 浏览器需要将域名（如 `www.example.com`）转换为服务器的 IP 地址。这个过程称为 ****DNS 解析****：
- a. 浏览器首先检查本地缓存，看是否已经缓存了该域名的 IP 地址。
 - b. 如果本地缓存中没有，浏览器会向 **ISP（互联网服务提供商）的 DNS 服务器** 发送 DNS 查询请求。

- c. 如果 ISP 的 DNS 服务器也没有缓存，它会向上级 DNS 服务器查询，直到找到根 DNS 服务器。
- d. 最终，DNS 解析返回目标服务器的 IP 地址。

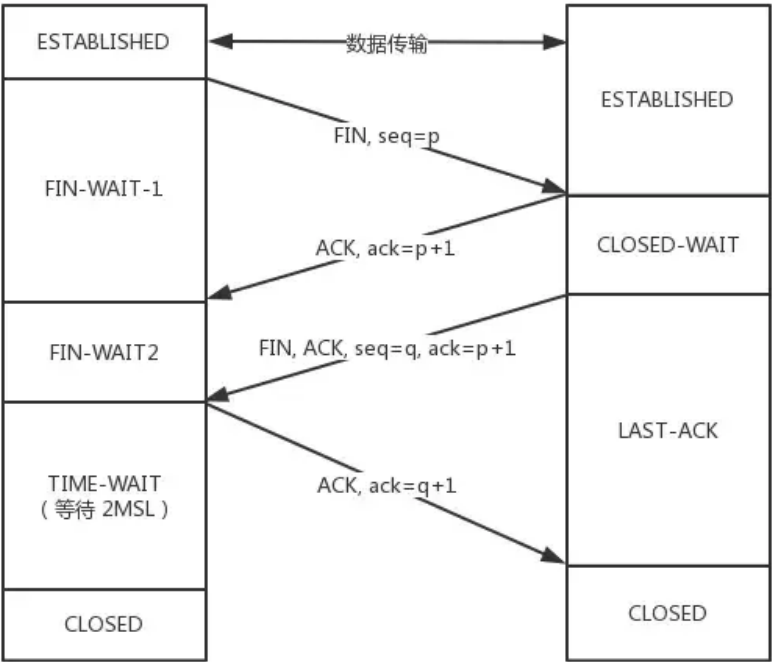
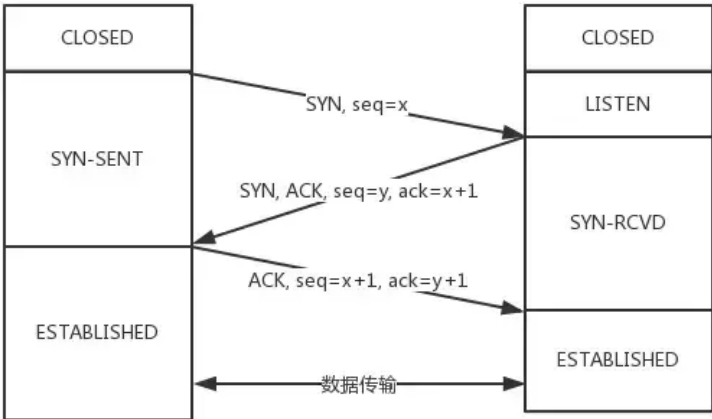
2. CDN

- a. DNS 服务器根据用户 IP 地址，将域名解析成相应节点的缓存服务器IP地址，实现用户就近访问

3. 建立网络连接

浏览器使用解析到的 IP 地址，与目标服务器建立网络连接：

- 1. **TCP 连接**：浏览器发起一个 TCP 三次握手过程，与服务器建立可靠的 TCP 连接。
- 2. **TLS/SSL 加密**：如果是 HTTPS 协议，浏览器和服务器会进行 TLS/SSL 握手，协商加密密钥，确保数据传输的安全。



4. 发送 HTTP/HTTPS 请求

浏览器通过已建立的连接，向服务器发送 HTTP 或 HTTPS 请求。请求包括：

- **请求行**：例如 `GET / HTTP/1.1`，表示请求根路径。
- **请求头**：包含浏览器信息（User-Agent）、语言偏好、请求来源（Referer）等。
- **请求体**（如果适用）：例如在 POST 请求中，包含表单数据或其他内容。

5. 服务器处理请求

服务器接收到请求后，根据请求内容进行处理：

6. 浏览器接收响应

服务器将响应通过网络发送回浏览器：

- 1. 浏览器接收完整的 HTTP 响应。
- 2. 浏览器解析响应头，根据 `Content-Type` 确定响应内容的类型（如 HTML、JSON、图片等）。

7. 渲染页面

如果响应内容是 HTML 页面，浏览器会进入渲染阶段：

- 1. **解析 HTML**：浏览器的渲染引擎（如 Blink、Gecko）解析 HTML 文档，构建 **DOM（文档对象模型）树**。
- 2. **解析 CSS**：浏览器解析页面中的 CSS 样式表，构建 **CSSOM（CSS 对象模型）树**。

- 3. **构建渲染树**：将 DOM 树和 CSSOM 树结合，生成渲染树，确定页面的布局和样式。
- 4. **布局（Reflow）**：计算每个元素的位置和大小。
- 5. **绘制（Paint）**：将渲染树转换为屏幕上的像素，显示页面内容。

8. 加载资源

HTML 页面可能包含其他资源（如图片、JavaScript 文件、CSS 文件等）：

- 1. 浏览器会解析 HTML 中的资源链接（如 `<img>`、`<script>`、`<link>`）。
- 2. 浏览器会为每个资源发起新的 HTTP 请求，下载资源。
- 3. 如果是 JavaScript 文件，浏览器会解析并执行脚本。
- 4. 如果是 CSS 文件，浏览器会重新解析样式并更新渲染树。

9. 交互与动态加载

用户与页面交互（如点击按钮、输入内容）可能触发 JavaScript 代码执行：

- 1. JavaScript 可能通过 AJAX 或 Fetch API 发起异步请求，获取新数据。
- 2. 浏览器会根据 JavaScript 的操作动态更新 DOM，重新渲染页面的局部内容。

2.14 前端 job model

P6 单业务执行

模块级优化与问题解决

技术能力

- **技术深度**：熟练使用主流框架（React/Vue），掌握前端工程化（Webpack/Vite），能优化性能、解决复杂业务问题。
- **代码能力**：独立完成复杂模块开发，具备良好的代码架构设计能力。
- **技术广度**：了解 Node.js、微前端等扩展技术，但主要聚焦业务开发。

业务能力

- **执行落地**：负责具体业务模块的开发，能理解产品需求并提出优化建议。
- **协作能力**：带领小型项目，指导 P5 及以下成员，推动团队内部技术改进。
- **影响力**：主要在团队内部，较少跨部门协作。

P7 跨业务技术赋能

识别业务痛点，技术驱动效率/增长
将技术转化为可量化的商业结果
从“执行者”到“规划者+推动者”

技术能力

- **架构能力**：能设计系统级技术方案，如微服务化、性能优化（如 Weex/HTTPS 升级案例）。
- **前瞻性**：推动新技术落地（如低代码、Serverless），解决公司级技术瓶颈。
- **技术领导力**：主导技术选型，确保架构长期可维护（如“1年后无需重构”标准）。

业务能力

- **业务规划**：深度参与业务决策，识别技术对业务的杠杆作用（如提升秒开率 92-95%）。
- **跨团队协作**：推动跨部门项目（如前端与客户端协作 Weex 化），影响业务技术方向。
- **团队管理**：通常带 4-5 人小团队，部分 P7 开始承担 TL（Team Leader）职责。

P8 战略级业务驱动

精通团队相关技术业务：关注业务的整体情况管理

技术能力

- **战略级架构**：主导公司级前端基建（如统一工程化体系、Flutter/Node 技术布局）。
- **行业影响力**：参与开源项目、技术大会分享，推动行业标准（如 Rax 体系）。
- **技术创新**：探索前沿技术（如 PWA、Service Worker），并判断其业务适用性。

业务能力

- **业务战略**：参与业务顶层设计，技术驱动业务增长（如 10亿+ HTTP 业务 HTTPS 化）。
- **团队规模**：管理 10-30 人团队，负责 HC 分配、绩效考核。
- **商业价值**：技术方案直接影响公司核心指标（如降低运维成本、提升开发效率）。

2.15 Docker

2.15.1 Docker 常见问题

1. 安装与启动问题

- a. **Cannot connect to the Docker daemon** 原因: Docker 服务未启动或用户权限不足**解决:**

Code block

```
1 sudo systemctl start docker      # 启动服务
2 sudo usermod -aG docker $USER    # 将用户加入 docker 组
3 newgrp docker                    # 刷新用户组
```

- b. **Failed to start Docker Application Container Engine** 原因: 系统不兼容或内核配置问题**解决:**

Code block

```
1 uname -r                        # 确认内核版本 > 3.10
2 sudo apt install linux-modules-extra-$(uname -r) # 安装内核扩展
```

2. 镜像问题

- a. **拉取镜像超时**原因: 网络问题或仓库访问慢**解决:**
- 配置镜像加速器（创建 `/etc/docker/daemon.json`）：

Code block

```
1 {
2   "registry-mirrors": [
3     "https://registry.docker-cn.com",
4     "https://docker.mirrors.ustc.edu.cn"
5   ]
6 }
```

- 重启服务: `sudo systemctl restart docker`
- b. **镜像占用磁盘过大**清理:

Code block

```
1 docker image prune -a      # 删除所有未使用的镜像
2 docker system prune       # 清理整个系统 (镜像+容器+网络)
```

c. 容器问题

- i. **容器启动后立即退出**原因: 容器内没有前台进程**解决:**
- 添加前台进程（如 Nginx）：

2.15.2 Docker 内网穿透

、工具/脚本/资源/案例

方案	适用场景	优点	缺点
FRP (Fast Reverse Proxy)	生产环境长期使用	高性能、配置灵活、支持多种协议	需自备公网服务器
Ngrok	临时测试/演示	无需自备服务器、简单易用	免费版不稳定、有连接限制
Cloudflare Tunnel	企业级应用	免公网IP、集成CDN和安全防护	依赖Cloudflare服务
SSH 隧道	临时访问/紧急调试	无需额外工具、所有系统支持	性能较差、不稳定

1. FRP frps.ini

- 公网配置

Code block

```
1 [common]
2 bind_port = 7000
3 vhost_http_port = 8080
4 token = your_secure_token
```

```
docker run -d --name frps \-p 7000:7000 -p 8080:8080 \-v /path/to/frps.ini:/etc/frp/frps.ini \-snowdreamtech/frps
```

- 内网配置

Code block

```
1 [common]
2 server_addr = your_public_ip
3 server_port = 7000
4 token = your_secure_token
5
6 [web]
7 type = http
8 local_port = 80
9 custom_domains = yourdomain.com
10
11 [ssh]
12 type = tcp
13 local_ip = 127.0.0.1
14 local_port = 22
15 remote_port = 6000
```

2. SSH 隧道

不安全，不建议用

Code block

```
1 docker run -d --name myapp \-p 8080:80 \
2   nginx:alpine
3
4 docker run -d --name frpc \--link myapp:myapp \-v
5   /path/to/frpc.ini:/etc/frp/frpc.ini \-snowdreamtech/frpc
```



```
1 CMD ["nginx", "-g", "daemon off;"] #  
Dockerfile 示例
```

- 启动时保持交互模式：

```
Code block  
1 docker run -it --entrypoint /bin/sh my-  
image
```

3. 网络问题

- a. 容器无法访问外网原因：DNS 配置错误解决：

```
Code block  
1 # 检查容器 DNS 配置  
2 docker run --dns 8.8.8.8 alpine ping  
google.com
```

- 永久修改：在 `/etc/docker/daemon.json` 中添加：

```
Code block  
1 { "dns": ["8.8.8.8", "114.114.114.114"] }
```

4. 存储问题

- a. 磁盘空间不足清理：

```
Code block  
1 docker system df # 查看磁盘  
占用  
2 docker volume prune # 删除未使  
用的数据卷  
3 docker builder prune # 清理构建  
缓存
```

5. 性能问题

- a. 容器 CPU/内存占用过高排查：

```
Code block  
1 docker stats # 实时监控资  
源  
2 docker top container_id # 查看容器内  
进程
```

限制资源：

```
Code block  
1 docker run --cpus=0.5 --memory=512m my-  
image
```

6. 安全与权限

- a. 容器内无法执行 `sudo`原因：默认以 `root` 运行推荐方案：
 - Dockerfile 中创建普通用户：

```
Code block
1  RUN useradd -ms /bin/bash appuser
2  USER appuser
```

7. 最佳实践建议

- a. 容器设计原则：单进程模型、无状态化、日志输出到 `stdout`
- b. 镜像优化：多阶段构建、减少层数、使用 `.dockerignore`
- c. 网络安全：限制容器间通信（`--network none`）、启用内容信任（DCT）
- d. 存储选择：
 - 临时数据 → `tmpfs`
 - 持久化数据 → `named volume`
 - 主机交互 → `bind mount`

2.16 事务

AICD

原子性 (Atomicity)

- 定义：事务是不可分割的工作单位，事务中的操作要么全部成功，要么全部失败回滚
- 实现机制：通过`undo log`（回滚日志）实现
- 示例：银行转账操作中，扣款和收款必须同时成功或同时失败

一致性 (Consistency)

- 定义：事务执行前后，数据库从一个一致状态变到另一个一致状态
- 特点：
 - 由其他三个特性共同保证
 - 包括数据完整性约束（如主键、外键等）
- 示例：转账前后，两个账户的总金额保持不变

隔离性 (Isolation)

- 定义：多个事务并发执行时，一个事务的执行不应影响其他事务的执行
- 实现机制：通过锁机制或MVCC（多版本并发控制）实现
- 问题：未充分隔离可能导致脏读、不可重复读、幻读

持久性 (Durability)

- 定义：事务一旦提交，其结果就是永久性的
- 实现机制：通过`redo log`（重做日志）实现
- 示例：事务提交后即使系统崩溃，数据也不会丢失

2.17 事务锁的方式

基本锁

简单易用，设置过期时间

```
SET key value NX PX timeout
```

```
Code block
1  const Redis = require('ioredis');
2  const redis = new Redis();
3
```

乐观锁

基于版本控制的非阻塞锁

```
WATCH + MULTI / EXEC
```

```
Code block
1  async function optimisticTransfer(fromAcc,
    toAcc, amount) {
2      const fromKey = `account:${fromAcc}`;
```

```

4  // 获取锁
5  async function acquireLock(lockName, timeout
    = 10000) {
6      const identifier =
    Math.random().toString(36).substr(2, 10);
7      const result = await
    redis.set(`lock:${lockName}`, identifier,
    'PX', timeout, 'NX');
8      return result === 'OK' ? identifier : null;
9  }
10
11 // 释放锁
12 async function releaseLock(lockName,
    identifier) {
13     const luaScript = `
14         if redis.call("get", KEYS[1]) == ARGV[1]
    then
15             return redis.call("del", KEYS[1])
16         else
17             return 0
18         end
19     `;
20     return await redis.eval(luaScript, 1,
    `lock:${lockName}`, identifier);
21 }
22
23 // 使用示例
24 async function businessOperation() {
25     const lockName = 'order_123';
26     const lockId = await acquireLock(lockName);
27
28     if (!lockId) {
29         console.log('获取锁失败');
30         return;
31     }
32
33     try {
34         // 执行业务逻辑
35         console.log('执行业务操作...');
36         await new Promise(resolve =>
    setTimeout(resolve, 2000));
37     } finally {
38         await releaseLock(lockName, lockId);
39     }
40 }

```

读写锁

多个键标记读/写状态 读共享，写互斥

Code block

```

1  async function acquireReadLock(lockName,
    clientId, timeout = 5000) {
2      const readKey = `rwlock:${lockName}:read`;
3      const writeKey = `rwlock:${lockName}:write`;
4
5      const luaScript = `
6          local writeKey = KEYS[1]
7          local readKey = KEYS[2]
8          local clientId = ARGV[1]

```

```

3      const toKey = `account:${toAcc}`;
4
5      await redis.watch(fromKey, toKey);
6
7      const balance = parseInt(await
    redis.get(fromKey)) || 0;
8      if (balance < amount) {
9          await redis.unwatch();
10         throw new Error('Insufficient balance');
11     }
12
13     const multi = redis.multi();
14     multi.decrby(fromKey, amount);
15     multi.incrby(toKey, amount);
16
17     const results = await multi.exec();
18
19     if (!results) {
20         throw new Error('Transaction failed due
    to concurrent modification');
21     }
22
23     return results;
24 }

```

RedLock 多节点

分布式高可用锁

Code block

```

1  const Redlock = require('redlock');
2  const redisInstances = [
3      new Redis(6379, 'redis1.example.com'),
4      new Redis(6379, 'redis2.example.com'),
5      new Redis(6379, 'redis3.example.com')
6  ];
7
8  const redlock = new Redlock(
9      redisInstances,

```

```

9     local timeout = ARGV[2]
10
11     if redis.call('exists', writeKey) == 1
12     then
13         return 0
14     end
15
16     redis.call('zadd', readKey, timeout,
17     clientId)
18     redis.call('pexpire', readKey, timeout)
19     return 1
20 `;
21
22 return await redis.eval(
23     luaScript,
24     2,
25     writeKey,
26     readKey,
27     clientId,
28     Date.now() + timeout
29 );
30 }
31
32 async function acquireWriteLock(lockName,
33     clientId, timeout = 5000) {
34     const readKey = `rwlock:${lockName}:read`;
35     const writeKey = `rwlock:${lockName}:write`;
36
37     const luaScript = `
38         local writeKey = KEYS[1]
39         local readKey = KEYS[2]
40         local clientId = ARGV[1]
41         local timeout = ARGV[2]
42
43         if redis.call('exists', writeKey) == 1
44         then
45             return 0
46         end
47
48         if redis.call('zcard', readKey) > 0 then
49             return 0
50         end
51
52         redis.call('set', writeKey, clientId,
53         'PX', timeout)
54         return 1
55     `;
56
57     return await redis.eval(
58         luaScript,
59         2,
60         writeKey,
61         readKey,
62         clientId,
63         timeout
64     );
65 }

```

```

10 {
11     driftFactor: 0.01,
12     retryCount: 3,
13     retryDelay: 200,
14     retryJitter: 200
15 }
16 );
17
18 async function criticalOperation() {
19     let lock;
20     try {
21         // 尝试获取锁 (至少2个节点同意)
22         lock = await redlock.lock('critical-
23         resource', 10000);
24
25         // 执行关键操作
26         console.log('执行关键操作...');
27         await performCriticalWork();
28     } catch (err) {
29         console.error('锁操作失败:', err);
30     } finally {
31         if (lock) {
32             await lock.unlock();
33         }
34     }
35 }

```

优化方式

1. 锁粒度：尽可能缩小锁的范围

- 2. 超时时间：设置合理的超时避免死锁
- 3. 本地缓存：对于频繁访问的锁可加入本地缓存
- 4. 锁分段：将一个大锁拆分为多个小锁
- 5. 避免锁嵌套：尽量减少锁的嵌套层级

2.18 基于 mastra搭建的MCP/A2A服务

2.18.1 MCP

1. 创建服务结构

Code block

```
1  import { Server } from
   "@modelcontextprotocol/sdk/server";
2  import express from 'express';
3
4  const app = express();
5  app.use(express.json());
6
7  // 初始化MCP服务器
8  const mcpServer = new Server({
9    name: "mastra-demo-service",
10   version: "1.0.0"
11 });
12
13 // 示例工具：获取当前时间
14 mcpServer.defineTool({
15   name: "getCurrentTime",
16   description: "获取当前服务器时间",
17   inputSchema: {
18     type: "object",
19     properties: {
20       timezone: {
21         type: "string",
22         description: "时区, 如Asia/Shanghai",
23         default: "UTC"
24       }
25     }
26   },
27   handler: async ({ timezone = "UTC" }) => {
28     return {
29       time: new Date().toLocaleString("en-US", { timeZone: timezone }),
30       timezone
31     };
32   }
33 });
34
35 // 启动HTTP服务器
36 const port = process.env.PORT || 3000;
37 app.listen(port, () => {
38   console.log(`MCP服务运行在
   http://localhost:${port}`);
39   mcpServer.start();
40 });
```

2. 添加方法tool

Code block

```
1  // 添加天气查询工具
2  mcpServer.defineTool({
3    name: "getWeather",
4    description: "获取指定城市的天气信息",
5    inputSchema: {
6      type: "object",
7      properties: {
8        location: {
9          type: "string",
10         description: "城市名称, 如Beijing"
11       },
12       unit: {
13         type: "string",
14         enum: ["celsius", "fahrenheit"],
15         default: "celsius"
16       }
17     },
18     required: ["location"]
19   },
20   handler: async ({ location, unit = "celsius"
   }) => {
21     // 这里可以集成真实天气API
22     return {
23       location,
24       temperature: unit === "celsius" ? 22 :
   72,
25       conditions: "Sunny",
26       updatedAt: new Date().toISOString()
27     };
28   }
29 });
30
31 // 添加计算工具
32 mcpServer.defineTool({
33   name: "calculate",
34   description: "执行数学计算",
35   inputSchema: {
36     type: "object",
37     properties: {
38       expression: {
39         type: "string",
40         description: "数学表达式, 如(10+5)*3"
41       }
42     },
43     required: ["expression"]
44   },
45   handler: async ({ expression }) => {
```



```
46     try {
47         return {
48             result: eval(expression), // 注意：生产
环境应使用更安全的计算方式
49             expression
50         };
51     } catch (error) {
52         return {
53             error: "无效的数学表达式",
54             details: error.message
55         };
56     }
57 }
58 }));
```

3. 能力集成

中间件 认证

Code block

```
1  import { MastraAuth } from 'mastra-auth-sdk';
2
3  // 初始化Mastra认证
4  const mastraAuth = new MastraAuth({
5      appId: process.env.MASTRA_APP_ID,
6      appSecret: process.env.MASTRA_APP_SECRET
7  });
8
9  // 添加认证中间件
10 app.use('/mcp', mastraAuth.authenticate());
```

暴露MCP发现端点

Code block

```
1  // 暴露MCP发现端点
2  app.get('/mcp/discovery', (req, res) => {
3      res.json({
4          protocol: "MCP-1.0",
5          tools: mcpServer.getToolNames(),
6          metadata: {
7              provider: "Mastra Demo Service",
8              version: "1.0.0"
9          }
10     });
11 });
12
13 // 暴露MCP执行端点
14 app.post('/mcp/execute', async (req, res) => {
15     const { tool, input } = req.body;
16     try {
17         const result = await
mcpServer.executeTool(tool, input);
18         res.json({ success: true, result });
19     } catch (error) {
20         res.status(400).json({
21             success: false,
22             error: error.message
23         });
24     }
25 });
```

4. docker部署

Code block

```
1  FROM node:18-alpine
2
3  WORKDIR /app
4  COPY package*.json ./
5  RUN npm install
6
7  COPY . .
8  RUN npm run build
```

Code block

```
1  docker build -t mastra-mcp-service .
2  docker run -p 3000:3000 -d mastra-mcp-service
```

```
9
10  ENV  PORT=3000
11  EXPOSE 3000
12
13  CMD ["node", "dist/server.js"]
```

5. 流式响应

Code block

```
1  mcpServer.defineTool({
2    name: "longRunningTask",
3    description: "长时间运行的任务，支持进度更新",
4    inputSchema: {
5      type: "object",
6      properties: {
7        steps: { type: "number", default: 5 }
8      }
9    },
10   handler: async ({ steps = 5 }, {
11     sendProgress }) => {
12     for (let i = 1; i <= steps; i++) {
13       await new Promise(resolve =>
14         setTimeout(resolve, 1000));
15       sendProgress({
16         progress: (i / steps) * 100,
17         message: `已完成步骤 ${i}/${steps}`
18       });
19     }
20   return { status: "completed", steps };
21 });
```

6. 集成外部API

Code block

```
1  mcpServer.defineTool({
2    name: "stockHistory",
3    description: "获取股票历史数据",
4    inputSchema: {
5      type: "object",
6      properties: {
7        symbol: { type: "string", description:
8          "股票代码，如TSLA" },
9        days: { type: "number", default: 30 }
10     },
11     required: ["symbol"]
12   },
13   handler: async ({ symbol, days = 30 }) => {
14     const response = await fetch(
15       `https://api.example.com/stocks/${symbol}?
16       days=${days}`,
17       { headers: { Authorization: `Bearer
18         ${process.env.STOCK_API_KEY}` } }
19     );
20     return response.json();
21   });
```

7. 权限管理

Code block

```
1  mcpServer.defineTool({
2    name: "adminOperation",
3    description: "需要管理员权限的操作",
4    permissions: ['admin'], handler: async
5    (input, { user }) => {if
6    (!user.roles.includes('admin')) {throw new
7    Error('无权访问此操作');} // 管理员操作逻辑...});
```

8. 速率限制

Code block

```
1  import rateLimit from 'express-rate-limit';
2
3  // 全局速率限制
4  const limiter = rateLimit({
5    windowMs: 15 * 60 * 1000, // 15分钟
6    max: 100 // 每个IP限制100次请求
7  });
8
9  app.use(limiter);
10
11  // 关键端点更严格的限制
12  app.use('/mcp/execute', rateLimit({
13    windowMs: 60 * 1000,
14    max: 10
15  }));
```

9. Prometheus监控

Code block

```
1  import client from 'prom-client';
2
```

10. 日志集中化

Code block

```
1  import winston from 'winston';
2  import { LokiTransport } from 'winston-loki';
```

```

3  const collectDefaultMetrics =
  client.collectDefaultMetrics;
4  collectDefaultMetrics({ timeout: 5000 });
5
6  app.get('/metrics', async (req, res) => {
7    res.set('Content-Type',
  client.register.contentType);
8    res.end(await client.register.metrics());
9  });
10
11  // 自定义指标
12  const mcpRequests = new client.Counter({
13    name: 'mcp_requests_total',
14    help: 'Total MCP tool requests',
15    labelNames: ['tool', 'status']
16  });

```

```

3
4  const logger = winston.createLogger({
5    transports: [
6      new LokiTransport({
7        host: process.env.LOKI_URL,
8        labels: { app: 'mastra-mcp-service' }
9      }),
10     new winston.transports.Console()
11   ]
12 });
13
14  // 在工具处理中使用
15  handler: async (input) => {
16    logger.info('处理工具请求', { tool:
  'getWeather', input });
17    // ...
18  }

```

2.18.9 A2A

- 代理(Agent)：具有特定能力的自治AI实体
- 通信协议：基于Mastra优化的WebSocket/HTTP混合协议
- 服务发现：通过Mastra Registry进行代理注册和发现

Node.js (18+) + TypeScript + Mastra SDK + WebSocket

1. 基础A2A代理

Code block

```

1  // src/agent.ts
2  import { MastraAgent } from '@mastra/a2a-core';
3  import WebSocket from 'ws';
4
5  export class MyAgent extends MastraAgent {
6    private wsServer: WebSocket.Server;
7
8    constructor(config: { port: number }) {
9      super({
10        agentId: 'my-service-agent',
11        capabilities: ['data-processing', 'decision-support']
12      });
13
14      this.wsServer = new WebSocket.Server({ port: config.port });
15      this.setupConnectionHandlers();
16    }
17
18    private setupConnectionHandlers() {
19      this.wsServer.on('connection', (ws) => {
20        ws.on('message', (data) => this.handleIncomingMessage(ws, data));
21        this.registerConnection(ws);
22      });
23    }
24
25    private async handleIncomingMessage(ws: WebSocket, data: WebSocket.Data) {
26      try {
27        const message = JSON.parse(data.toString());
28        const response = await this.processMessage(message);

```

```

29     ws.send(JSON.stringify(response));
30   } catch (error) {
31     ws.send(JSON.stringify({
32       error: 'INVALID_REQUEST',
33       details: error.message
34     }));
35   }
36 }
37 }
38
39 // 实现消息处理器
40 // src/messageHandlers.ts
41 import { z } from 'zod';
42
43 const RequestSchema = z.object({
44   action: z.enum(['analyze', 'transform', 'decide']),
45   payload: z.record(z.unknown()),
46   context: z.object({
47     sessionId: z.string(),
48     sourceAgent: z.string()
49   }).optional()
50 });
51
52 export async function processMessage(rawData: unknown) {
53   const { action, payload, context } = RequestSchema.parse(rawData);
54
55   switch (action) {
56     case 'analyze':
57       return handleAnalysis(payload);
58     case 'transform':
59       return handleTransformation(payload);
60     case 'decide':
61       return handleDecision(payload, context);
62     default:
63       throw new Error('UNSUPPORTED_ACTION');
64   }
65 }
66
67 async function handleAnalysis(payload: Record<string, unknown>) {
68   // 实现分析逻辑
69   return {
70     status: 'completed',
71     insights: []
72   };
73 }

```

2. Mastra平台集成

注册到mastra registry

Code block

```

1 // src/registry.ts
2 import { MastraRegistry } from '@mastra/a2a-core';
3
4 export async function registerAgent() {
5   const registry = new MastraRegistry({
6     apiKey: process.env.MASTRA_API_KEY,
7     endpoint: 'https://api.mastra.ai/registry/v1'
8   });
9

```

```

10     await registry.register({
11       agentId: 'my-service-agent',
12       endpoint: `ws://${process.env.HOST}:${process.env.PORT}`,
13       capabilities: ['data-processing', 'decision-support'],
14       status: 'available'
15     });
16   }
17
18   // 心跳检测
19   private heartbeatInterval: NodeJS.Timeout;
20
21   startHeartbeat() {
22     this.heartbeatInterval = setInterval(() => {
23       this.registry.updateStatus('active');
24     }, 30000); // 每30秒发送心跳
25   }
26
27   shutdown() {
28     clearInterval(this.heartbeatInterval);
29     this.registry.unregister();
30   }

```

3. docker部署

Code block

```

1   FROM node:18-alpine
2
3   WORKDIR /app
4   COPY package*.json ./
5   RUN npm install
6
7   COPY . .
8   RUN npm run build
9
10  ENV PORT=8080
11  EXPOSE 8080
12
13  CMD ["node", "dist/server.js"]

```

3. 多代理协作模式

Code block

```

1   // src/collaboration.ts
2   import { MastraOrchestrator } from '@mastra/a2a-core';
3
4   export async function handleComplexTask(task) {
5     const orchestrator = new MastraOrchestrator();
6
7     const [dataAgent, decisionAgent] = await Promise.all([
8       orchestrator.findAgent('data-processing'),
9       orchestrator.findAgent('decision-support')
10    ]);
11
12    const processed = await dataAgent.execute({
13      action: 'process',
14      payload: task.data
15    });
16

```



```
17     return decisionAgent.execute({
18       action: 'makeDecision',
19       payload: processed.result
20     });
21   }
```

4. 会话状态管理

Code block

```
1  // src/sessionManager.ts
2  import { createClient } from 'redis';
3
4  class SessionManager {
5    private client = createClient({
6      url: process.env.REDIS_URL
7    });
8
9    constructor() {
10      this.client.connect();
11    }
12
13    async trackSession(sessionId: string, context: Record<string, unknown>) {
14      await this.client.hSet(
15        `session:${sessionId}`,
16        'lastActive', Date.now(),
17        'context', JSON.stringify(context)
18      );
19    }
20  }
```

5. 监控

Code block

```
1  import client from 'prom-client';
2
3  const agentRequests = new client.Counter({
4    name: 'a2a_requests_total',
5    help: 'Total A2A requests processed',
6    labelNames: ['action', 'status']
7  });
8
9  // 在消息处理器中
10 agentRequests.inc({
11   action: message.action,
12   status: 'success'
13 });
```

6. 结构化日志

Code block

```
1  import pino from 'pino';
2
3  const logger = pino({
4    level: 'info',
5    transport: {
6      target: 'pino-pretty',
7      options: {
```

```
8         colorize: true
9     }
10 }
11 });
12
13 // 使用示例
14 logger.info({
15     event: 'agent_request',
16     action: message.action,
17     duration: `${Date.now() - start}ms`
18 });
```

3. 完整的前端技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



4. 技术面试（高阶终面）注意事项

1. 技术终面准备

主要是对Owner意识的考察与技术考察。

1. 在技术领域，我觉得可能会被考察的题目主要集中在基础的扎实程度上。

2. 在意识层面，主要是下面几个方面：

a. 软性方面的考察

- i. 交流感觉：你带给面试官的整体感觉；
- ii. 对待事情的想法和冲劲：你面对具体事务时的态度和动力。

b. 项目方面的考察

- i. 项目方面，考察的是你是否具备前瞻性认知，包括以下几个方面：
 - 1. 资源布局想法：如何合理分配和利用资源；
 - 2. 规划能力：对项目整体规划的能力；
 - 3. 聪明做事：能否以更智慧的方式完成任务，而不仅仅是单纯编写代码；
 - 4. 协调资源：还需要考察你可能会如何考虑去协调资源和管理成员。

3. 主管层面的面试

a. 需要准备的内容：

- i. 核心项目讲述：准备至少一个自己作为主导的核心项目案例。这与职级无关，即使是P6也有自己负责的小项目，并且可以学习P7+及以上级别owner的业务思维。比如，可以参考学习团队内的核心mentor们是如何考虑资源问题、了解市场业务需求的。面试中讲述时，也应该包含项目的背景、接受的任务、布局思路等，而不仅仅是技术细节。
- ii. 软性选择逻辑：体现你的选择逻辑，即自己是否有明确的向上发展的意愿，是否愿意参与或主导更具挑战性的项目。面试中，你需要清晰地表达自己下一阶段的目标和所需要的一些资源。（意识表现是关键！）

b. 面试官的关注点：项目经验60%+选择逻辑20%+意愿度20%

c. 面试中的主动性：

- i. 在面试中，无论经验多少，都不能只是被动等待面试官提问，需要学会主动掌控节奏。（这一点非常重要！）

4. 提问互动环节

在提问互动环节，候选人至少要提出一个问题，例如：

- a. 我们这边主要遇到了什么样的业务和技术问题？
- b. 需要人选什么样的能力？
- c. 面试官对我的期望是什么？

还有更聪明的做法是，可以在面试官回答咱需要的能力时，主动结合自己过去的经验来表现说明：自己之前是如何解决类似问题的，或者说自己可以复用的能力部分是什么？（说白了就是主动推荐自己，毛遂自荐。在面试场上，只有足够主动，才能争取到理想的定级～！）

5. 手写真题详解

5.1 两数之和

给定一个整数数组 `nums` 和一个目标值 `target`，请你在该数组中找出和为目标值的那 **两个** 整数，并返回他们的数组下标。

你可以假设每种输入只会对应一个答案。但是，你不能重复利用这个数组中同样的元素。

示例：

Code block

```
1  给定 nums = [2, 7, 11, 15], target = 9
```

```
2  因为 nums[0] + nums[1] = 2 + 7 = 9所以返回 [0, 1]
```

解题思路：

- 初始化一个 `map = new Map()`
- 从第一个元素开始遍历 `nums`
- 获取目标值与 `nums[i]` 的差值，即 `k = target - nums[i]`，判断差值在 `map` 中是否存在
 - 不存在（`map.has(k)` 为 `false`），则将 `nums[i]` 加入到 `map` 中（key为 `nums[i]`，value为 `i`，方便查找 `map` 中是否存在某值，并可以通过 `get` 方法直接拿到下标）
 - 存在（`map.has(k)`），返回 `[map.get(k), i]`，求解结束
- 遍历结束，则 `nums` 中没有符合条件的两个数，返回 `[]`

时间复杂度： $O(n)$

代码实现：

Code block

```
1  const twoSum = function(nums, target) {
2      let map = new Map()
3      for(let i = 0; i < nums.length; i++) {
4          let k = target - nums[i]
5          if(map.has(k)) {
6              return [map.get(k), i]
7          }
8          map.set(nums[i], i)
9      }
10     return [];
11 };
```

5.2 N数之和

请用算法实现，从给定的无需、不重复的数组A中，取出N个数，使其相加和为M。并给出算法的时间、空间复杂度，如：

Code block

```
1  var arr = [1, 4, 7, 11, 9, 8, 10, 6];
2  var N = 3;
3  var M = 27;
4
5  Result:
6  [7, 11, 9], [11, 10, 6], [9, 8, 10]
```

解题思路：利用二进制

根据数组长度构建二进制数据，再选择其中满足条件的数据。

我们用 `1` 和 `0` 来表示数组中某位元素是否被选中。因此，可以用 `0110` 来表示数组中第 `1` 位和第 `2` 位被选中了。

所以，本题可以解读为：

- 数组中被选中的个数是 `N`。
- 被选中的和是 `M`。

我们的算法思路逐渐清晰起来：遍历所有二进制，判断选中个数是否为 `N`，然后再求对应的元素之和，看其是否为 `M`。

1. 从数组中取出 N 个数

例如：

```
1  let arr = [1, 2, 3, 4];
2  var N = 3;
3  var M = 6;
```

如何判断 $N=3$ 是，对应的选取二进制中有几个 1 喃？

最简单的方式就是：

Code block

```
1  const n = num => num.toString(2).replace(/0/g, '').length
```

这里我们尝试使用一下位运算来解决本题，因为位运算是不需要编译的（位运算直接用二进制进行表示，省去了中间过程的各种复杂转换，提高了速度），肯定速度最快。

我们知道 $1\&0=0$ 、 $1\&1=1$ ， $1111\&1110=1110$ ，即 $15\&14=14$ ，所以我们每次 $\&$ 比自身小 1 的数都会消除一个 1 ，所以这里建立一个迭代，通过统计消除的次数，就能确定最终有几个 1 了：

Code block

```
1  const n = num => {
2    let count = 0
3    while(num) {
4      num &= (num - 1)
5      count++
6    }
7    return count
8  }
```

2. 和为 M

现在最后一层判断就是选取的这些数字和必须等于 M ，即根据 N 生成的对应二进制所在元素上的和 是否为 M 。

比如 1110 ，我们应该判断 $arr[0] + arr[1] + arr[2]$ 是否为 M 。

那么问题也就转化成了如何判断数组下标是否在 1110 中呢？如何在则求和

其实也很简单，比如下标 1 在，而下标 3 不在。我们把 1 转化成 0100 ， $1110 \& 0100$ 为 0100 ,大于 0 ，因此下标 1 在。而 $1110 \& 0001$ 为 0 ，因此下标 3 不在。

所以求和我们可以如下实现：

Code block

```
1  let arr = [1,2,3,4]
2  // i 为满足条件的二进制
3  let i = 0b1110
4  let s = 0, temp = []
5  let len = arr.length
6  for (let j = 0; j < len; j++) {
7    if ( i & 1 << (len - 1 - j)) {
8      s += arr[j]
9      temp.push(arr[j])
10   }
11 }
12 console.log(temp)
13 // => [1, 2, 3]
```

最终实现

Code block


```

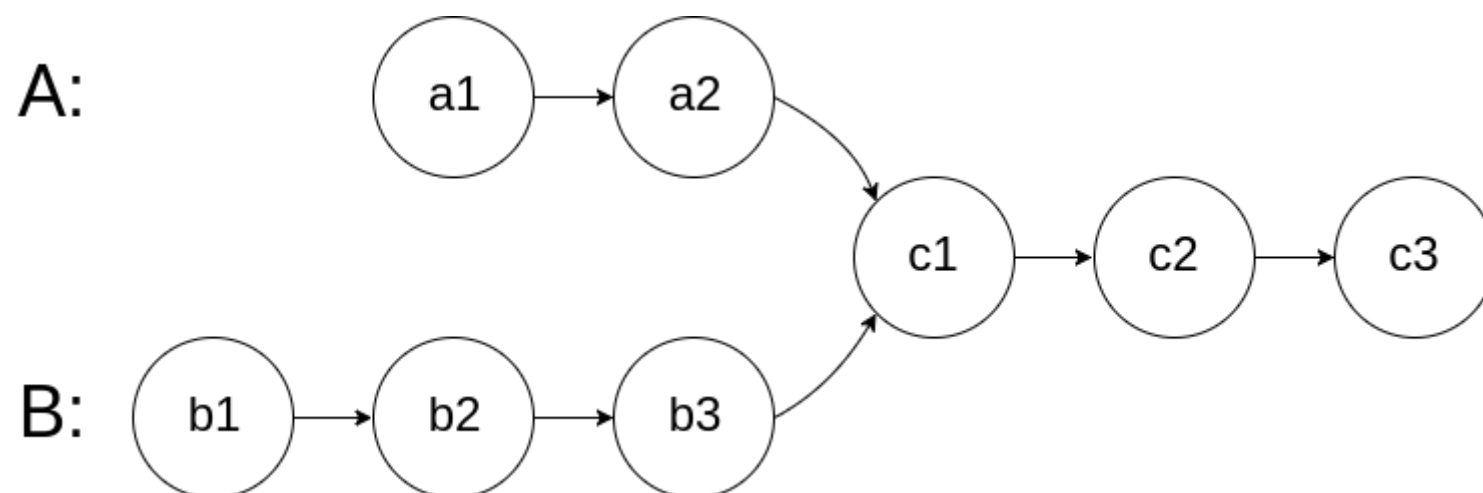
1  // 参数依次为目标数组、选取元素数目、目标和
2  const search = (arr, count, sum) => {
3    // 计算某选择情况下有几个 1，也就是选择元素的个数
4    const getCount = num => {
5      let count = 0
6      while(num) {
7        num &= (num - 1)
8        count++
9      }
10     return count
11   }
12
13   let len = arr.length, bit = 1 << len, res = []
14
15   // 遍历所有的选择情况
16   for(let i = 1; i < bit; i++){
17     // 满足选择的元素个数 === count
18     if(getCount(i) === count){
19       let s = 0, temp = []
20
21       // 每一种满足个数为 N 的选择情况下，继续判断是否满足 和为 M
22       for(let j = 0; j < len; j++){
23         // 建立映射，找出选择位上的元素
24         if(i & 1 << (len - 1 - j)) {
25           s += arr[j]
26           temp.push(arr[j])
27         }
28       }
29
30       // 如果这种选择情况满足和为 M
31       if(s === sum) {
32         res.push(temp)
33       }
34     }
35   }
36
37   return res
38 }

```

5.3 编写一个程序，找到两个单链表相交的起始节点

编写一个程序，找到两个单链表相交的起始节点。

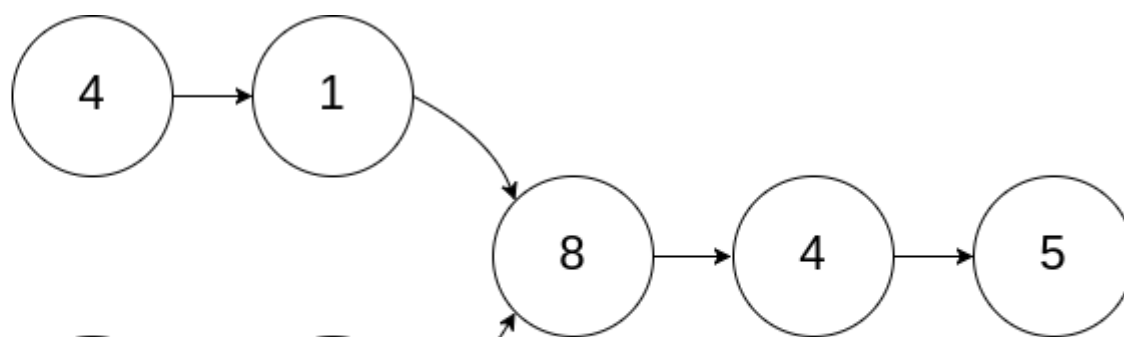
如下面的两个链表：



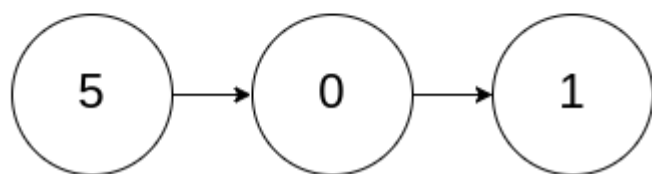
在节点 c1 开始相交。

示例 1:

A:



B:

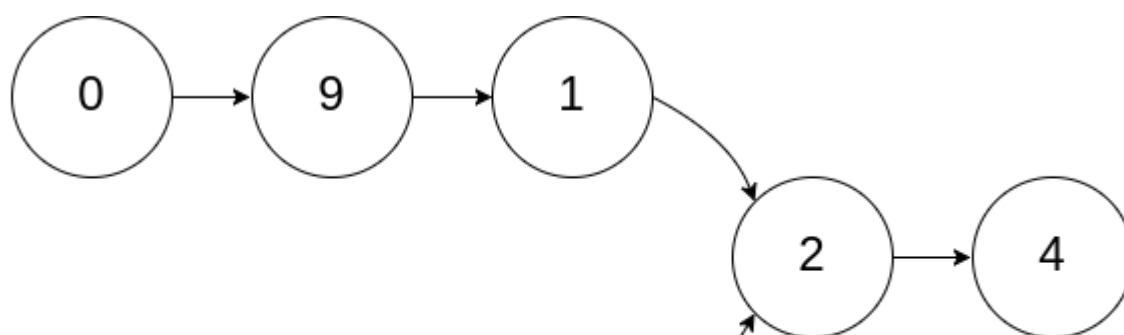


Code block

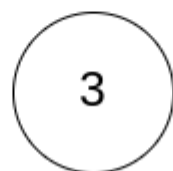
- 1 输入: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,0,1,8,4,5], skipA = 2, skipB = 3
- 2 输出: Reference of the node with value = 8
- 3 输入解释: 相交节点的值为 8 (注意, 如果两个列表相交则不能为 0)。从各自的表头开始算起, 链表 A 为 [4,1,8,4,5], 链表 B 为 [5,0,1,8,4,5]。在 A 中, 相交节点前有 2 个节点; 在 B 中, 相交节点前有 3 个节点。

示例 2:

A:



B:

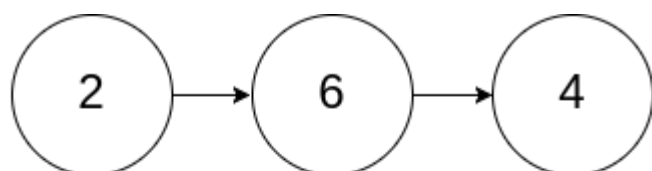


Code block

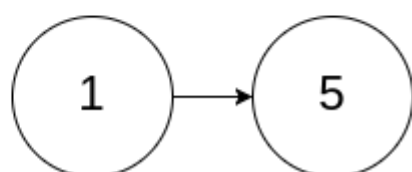
- 1 输入: intersectVal = 2, listA = [0,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
- 2 输出: Reference of the node with value = 2
- 3 输入解释: 相交节点的值为 2 (注意, 如果两个列表相交则不能为 0)。从各自的表头开始算起, 链表 A 为 [0,9,1,2,4], 链表 B 为 [3,2,4]。在 A 中, 相交节点前有 3 个节点; 在 B 中, 相交节点前有 1 个节点。

示例 3:

A:



B:



Code block

- 1 输入: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
- 2 输出: null
- 3 输入解释: 从各自的表头开始算起, 链表 A 为 [2,6,4], 链表 B 为 [1,5]。由于这两个链表不相交, 所以 intersectVal 必须为 0, 而 skipA 和 skipB 可以是任意值。
- 4 解释: 这两个链表不相交, 因此返回 null。

注意：

- 如果两个链表没有交点，返回 null.
- 在返回结果后，两个链表仍须保持原有的结构。
- 可假定整个链表结构中没有循环。
- 程序尽量满足 $O(n)$ 时间复杂度，且仅用 $O(1)$ 内存。

方法：遍历

- 先将两个链表转换成数组结构
- 对其中一个数组进行遍历，每次遍历判断另一个数组是否含有该值
- 遍历结束后如果没有找到就返回 null

Code block

```
1  function intersectNode( head1, head2 ) {
2      var arr1 = [],
3          arr2 = [],
4          theValue = null;
5      transformToArray( head1, arr1 );
6      transformToArray( head2, arr2 );
7
8      for (var i = 0; i < arr1.length; i++) {
9          if( arr2.indexOf(arr1[i]) !== -1 ){
10             theValue = arr1[i];
11             return 'Reference of the node with value = ' + theValue;
12         }
13     }
14     return theValue;
15
16     /*
17         arr1.some( value => {
18             if ( arr2.includes(value) ){
19                 theValue = value;
20                 return true;
21             }
22         })
23         if( theValue ){
24             return 'Reference of the node with value = ' + theValue;
25         }
26         return null;
27     */
28 }
29
30 function transformToArray( head, arr ) {
31     while( head ){
32         arr.push(head.val);
33         head = head.next;
34     }
35 }
```

测试代码：

Code block

```
1  function CreateNode(val){
2      this.val = val;
```

```

3      this.next = null;
4  }
5  function CreateList(...nodes){
6      this.head = nodes[0];
7      this.length = nodes.length;
8      for( var i = 0; i < nodes.length - 1; i++ ){
9          if( nodes[i+1] ){
10             nodes[i].next = nodes[i+1];
11         }
12     }
13 }
14
15 function intersectNode( head1, head2 ) {
16     var arr1 = [],
17         arr2 = [],
18         theValue = null;
19     transformToArray( head1, arr1 );
20     transformToArray( head2, arr2 );
21
22     for (var i = 0; i < arr1.length; i++) {
23         if( arr2.indexOf(arr1[i]) !== -1 ){
24             theValue = arr1[i];
25             return 'Reference of the node with value = ' + theValue;
26         }
27     }
28     return theValue;
29 }
30 function transformToArray( head, arr ) {
31     while( head ){
32         arr.push(head.val);
33         head = head.next;
34     }
35 }
36
37 const node1 = new CreateNode(1);
38 const node2 = new CreateNode(2);
39 const node3 = new CreateNode(3);
40 const node4 = new CreateNode(4);
41 const node5 = new CreateNode(5);
42 const nodeA = new CreateNode('A');
43 const nodeB = new CreateNode('B');
44 const nodeC = new CreateNode('C');
45 const nodeD = new CreateNode('D');
46 const nodeE = new CreateNode('E');
47 const list1 = new CreateList(node1, node2, node3, node4, node5);
48 const list2 = new CreateList(nodeA, nodeB, node3, nodeC, nodeD);
49
50 intersectNode(list1.head, list2.head);

```

5.4 翻转字符串里的单词

给定一个字符串，逐个翻转字符串中的每个单词。

示例 1:

Code block

```

1  输入: "the sky is blue"
2  输出: "blue is sky the"

```

示例 2：

Code block

- 1 输入：" hello world! "
- 2 输出："world! hello"
- 3 解释：输入字符串可以在前面或者后面包含多余的空格，但是反转后的字符不能包括。

示例 3：

Code block

- 1 输入："a good example"
- 2 输出："example good a"
- 3 解释：如果两个单词间有多余的空格，将反转后单词间的空格减少到只含一个。

说明：

- 无空格字符构成一个单词。
- 输入字符串可以在前面或者后面包含多余的空格，但是反转后的字符不能包括。
- 如果两个单词间有多余的空格，将反转后单词间的空格减少到只含一个。

解法一：正则 + JS API

Code block

- ```
1 var reverseWords = function(s) {
2 return s.trim().replace(/\s+/g, ' ').split(' ').reverse().join(' ');
3 };
```

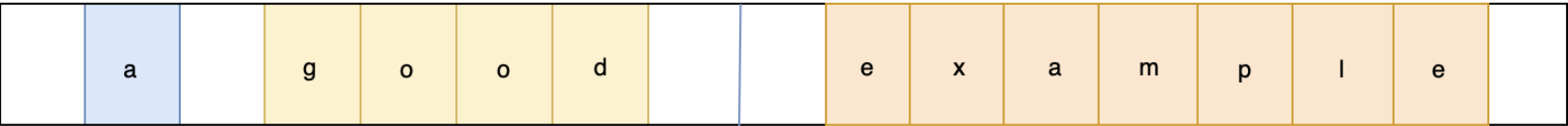
解法二：双端队列（不使用 API）

双端队列，故名思义就是两端都可以进队的队列

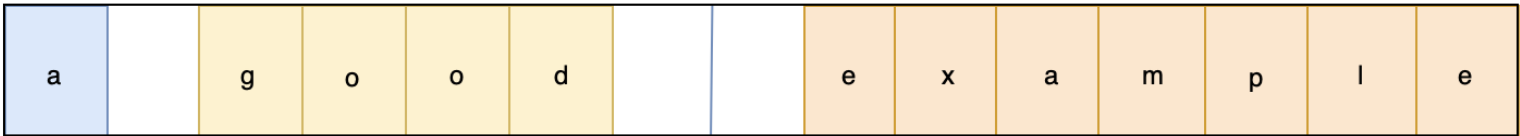
解题思路：

- 首先去除字符串左右空格
- 逐个读取字符串中的每个单词，依次放入双端队列的对头
- 再将队列转换成字符串输出（以空格为分隔符）

画图理解：



删除字符串两端的空格



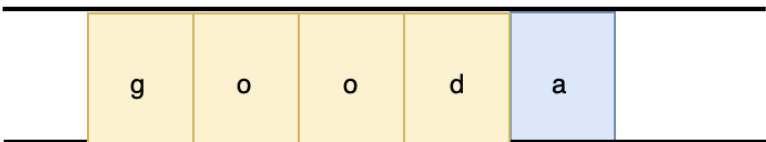
从左到右，依次读取单词



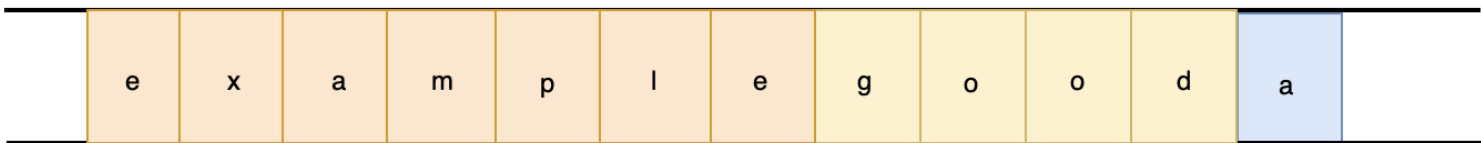
将第一个单词放入双端队列对头中



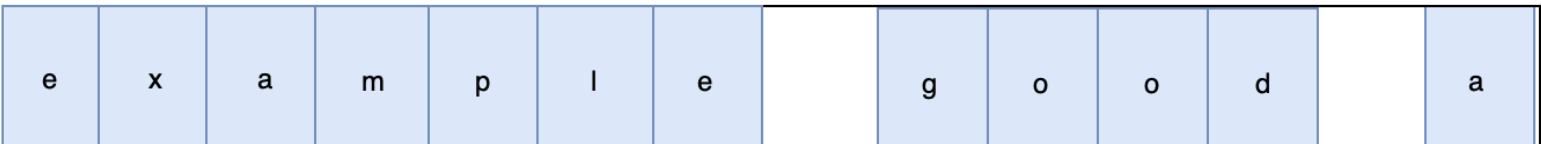
将第二个单词也双端队列对头中



依次，将所有的单词都放入双端队列队头中  
此时，队列中的单词顺序被翻转



将队列中的单词以字符串的形式输出  
并以空格作为分隔符





代码实现：

Code block

```
1 var reverseWords = function(s) {
2 let left = 0
3 let right = s.length - 1
4 let queue = []
5 let word = ''
6 while (s.charAt(left) === ' ') left ++
7 while (s.charAt(right) === ' ') right --
8 while (left <= right) {
9 let char = s.charAt(left)
10 if (char === ' ' && word) {
11 queue.unshift(word)
12 word = ''
13 } else if (char !== ' '){
14 word += char
15 }
16 left++
17 }
18 queue.unshift(word)
19 return queue.join(' ')
20 };
```

## 5.5 无重复字符的最长子串

示例 1:

Code block

```
1 输入: "abcabcbb"
2 输出: 3
3 解释: 因为无重复字符的最长子串是 "abc", 所以其长度为 3。
```

示例 2:

Code block

```
1 输入: "bbbbbb"
2 输出: 1
3 解释: 因为无重复字符的最长子串是 "b", 所以其长度为 1。
```

示例 3:

Code block

```
1 输入: "pwwkew"
2 输出: 3
3 解释: 因为无重复字符的最长子串是 "wke", 所以其长度为 3。请注意, 你的答案必须是 子串 的长度, "pwke" 是一个子序列, 不是子串。
```

快慢指针维护一个滑动窗口，如果滑动窗口里面有快指针fastp所指向的元素（记录这重复元素在滑动窗口的索引tmp），那么滑动窗口要缩小，即慢指针slowp要移动到tmp的后一个元素。

Code block

```
1 /**
2 * @param {string} s
```

```

3 * @return {number}
4 */
5 var lengthOfLongestSubstring = function(s) {
6 let res = 0, slowp = 0, tmp = 0
7 for(let fastp = 0; fastp < s.length; fastp++) {
8 tmp = s.substring(slowp, fastp).indexOf(s.charAt(fastp)) // 滑动窗口有没有重复元素
9 if(tmp === -1) { // 没有
10 res = Math.max(res, fastp-slowp+1) // 上一次值和滑动窗口值大小比较
11 }else{ // 有, 移动慢指针, 是slowp+tmp+1不是tmp+1, 因为tmp是根据滑动窗口算的
12 slowp = slowp + tmp + 1
13 }
14 }
15 return res
16 };

```

## 5.6 字符串相加

给定两个字符串形式的非负整数 `num1` 和 `num2`，计算它们的和。

例如：

"111" + "2222" = "2333"

注意：

- `num1` 和 `num2` 的长度都小于 5100
- `num1` 和 `num2` 都只包含数字 0-9
- `num1` 和 `num2` 都不包含任何前导零
- 你不能使用任何内建 `BigInteger` 库，也不能直接将输入的字符串转换为整数形式

### 从低位开始相加

- 从 `num1`，`num2` 的尾部开始计算，模拟人工加法，保存到 `tmp` 中；
- 计算 `tmp` 的个位数，并添加到 `result` 的头部，这里的 `result` 是 `string` 类型，不是 `number` 类型；
- 计算进位，改成 `tmp`，进行下次循环
- 索引溢出处理：循环结束，根据 `tmp` 判断是否有进位，并在 `result` 头部添加进位 1
- 返回 `result`

代码实现：

Code block

```

1 var addStrings = function(num1, num2) {
2 let a = num1.length, b = num2.length, result = '', tmp = 0
3 while(a || b) {
4 a ? tmp += +num1[--a] : ''
5 b ? tmp += +num2[--b] : ''
6
7 result = tmp % 10 + result
8 if(tmp > 9) tmp = 1
9 else tmp = 0
10 }
11 if (tmp) result = 1 + result
12 return result
13 };

```

复杂度分析：

- 时间复杂度  $O(\max(M,N))$
- 空间复杂度  $O(1)$

### 5.7 最小栈（包含getMin函数的栈）

设计一个支持 `push` ， `pop` ， `top` 操作，并能在常数时间内检索到最小元素的栈。

- `push(x)` —— 将元素  $x$  推入栈中。
- `pop()` —— 删除栈顶的元素。
- `top()` —— 获取栈顶元素。
- `getMin()` —— 检索栈中的最小元素。

Code block

```
1 MinStack minStack = new MinStack();
2 minStack.push(-2);
3 minStack.push(0);
4 minStack.push(-3);
5 minStack.getMin(); --> 返回 -3.
6 minStack.pop();minStack.top(); --> 返回 0.
7 minStack.getMin(); --> 返回 -2.
```

Code block

```
1 class MinStack {
2 constructor () {
3 this.length = 0;
4 this.content = [];
5 this.mins = [];
6 }
7 push (val) {
8 const curMin = this.mins[this.length - 1] !== undefined ? this.mins[this.length - 1] : Infinity;
9 this.content[this.length++] = val;
10 this.mins[this.length - 1] = Math.min(curMin, val);
11 }
12 pop () {
13 return this.content[--this.length]
14 }
15 top () {
16 return this.content[this.length - 1]
17 }
18 getMin () {
19 return this.mins[this.length - 1];
20 }
21 }
```

### 5.8 翻转字符串里的单词

给定一个字符串，逐个翻转字符串中的每个单词。

示例 1:

Code block

```
1 输入: "the sky is blue"
2 输出: "blue is sky the"
```

示例 2:

Code block

```
1 输入: " hello world! "
2 输出: "world! hello"
3 解释: 输入字符串可以在前面或者后面包含多余的空格, 但是反转后的字符不能包括。
```

### 示例 3:

Code block

```
1 输入: "a good example"
2 输出: "example good a"
3 解释: 如果两个单词间有多余的空格, 将反转后单词间的空格减少到只含一个。
```

### 说明:

- 无空格字符构成一个单词。
- 输入字符串可以在前面或者后面包含多余的空格, 但是反转后的字符不能包括。
- 如果两个单词间有多余的空格, 将反转后单词间的空格减少到只含一个。

### 方法一

Code block

```
1 const reverseWords = s => {
2 let arr = s.split(' ').filter(function (s) {
3 return s && s.trim();
4 })
5 const result = arr.reverse().join(' ')
6 return result;
7 };
```

### 方法二

Code block

```
1 const reverseWords = s => {
2 return s.match(/[^\w!,,]+/g).reverse().join(' ');
3 }
```

## 5.9 无重复字符的最长子串

给定一个字符串, 请你找出其中不含有重复字符的 **最长子串** 的长度。

### 示例 1:

Code block

```
1 输入: "abcabcbb"
2 输出: 3
3 解释: 因为无重复字符的最长子串是 "abc", 所以其长度为 3。
```

### 示例 2:

Code block

```
1 输入: "bbbbbb"
```

```
2 输出：1
3 解释：因为无重复字符的最长子串是 "b"，所以其长度为 1。
```

### 示例 3:

Code block

```
1 输入："pwwkew"
2 输出：3
3 解释：因为无重复字符的最长子串是 "wke"，所以其长度为 3。请注意，你的答案必须是 子串 的长度，"pwke" 是一个子序列，不是子串。
```

快慢指针维护一个滑动窗口，如果滑动窗口里面有快指针fastp所指向的元素（记录这重复元素在滑动窗口的索引tmp），那么滑动窗口要缩小，即慢指针slowp要移动到tmp的后一个元素。

Code block

```
1 /**
2 * @param {string} s
3 * @return {number}
4 */
5 var lengthOfLongestSubstring = function(s) {
6 let res = 0, slowp = 0, tmp = 0
7 for(let fastp = 0; fastp < s.length; fastp++) {
8 tmp = s.substring(slowp, fastp).indexOf(s.charAt(fastp)) // 滑动窗口有没有重复元素
9 if(tmp === -1) { // 没有
10 res = Math.max(res, fastp-slowp+1) // 上一次值和滑动窗口值大小比较
11 }else{ // 有，移动慢指针， 是slowp+tmp+1不是tmp+1，因为tmp是根据滑动窗口算的
12 slowp = slowp + tmp + 1
13 }
14 }
15 return res
16 };
```